

Introduction to SQL

SQL (Structured Query Language) is a standard programming language used to create, manage, and manipulate relational databases. It allows you to store, retrieve, update, and delete data efficiently.

Key Features



Easy to Learn

Simple and readable syntax.



Standard Language

ANSI/ISO standard used worldwide.



Powerful

Handles complex queries with ease.



Relational

Works with data in tables and relations.



Secure

Role-based access and permissions.



How SQL Works



You write SQL queries



Database processes the query



Results are returned



You use the results

Common SQL Commands

SELECT	Retrieve data from one or more tables.
INSERT	Insert new data into a table.
UPDATE	Modify existing data in a table.
DELETE	Remove data from a table.
CREATE	Create a new database, table, or object.
ALTER	Modify the structure of an existing object.
DROP	Delete an existing object.

Example

SQL Query

```
SELECT name, email
FROM users
WHERE age > 18
ORDER BY name;
```



Result

name	email
Alice	alice@example.com
Bob	bob@example.com
Charlie	charlie@example.com

Common Use Cases



Data Retrieval & Reporting



Data Analysis



Data Management



Access Control & Security



Application Development



SQL is the foundation of data-driven applications and a must-have skill for **developers**, **data analysts**, and **database administrators**.

Introduction to MySQL

MySQL is an [open-source relational database management system](#) (RDBMS) based on SQL. It is reliable, fast, secure, and widely used for web applications and enterprise solutions.

Key Features



High Performance



Reliable & Secure



Scalable



Open Source



Easy to Use

Common Use Cases



Web Applications



E-commerce Platforms



Data Storage & Analytics



Content Management



Enterprise Applications



Example

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  created_at DATETIME  
);  
  
SELECT * FROM users;
```

id	name	email	created_at
1	Alice	alice@example.com	2024-01-01 10:00:00
2	Bob	bob@example.com	2024-01-01 11:15:00
3	Charlie	charlie@example.com	2024-01-01 12:30:00



MySQL powers many of the world's most popular websites and applications, including [Facebook](#), [Twitter](#), [YouTube](#), and [WordPress](#).

ACID Properties in MySQL



ACID properties ensure that database transactions are reliable, consistent, and safe even in case of errors or system failures.



A Atomicity

All operations in a transaction are treated as a single unit. Either all succeed, or none are applied.



Example

If a fund transfer fails in the middle, no amount is deducted or added.



In MySQL

InnoDB ensures atomicity by using rollback to undo all changes if any error occurs.



C Consistency

A transaction brings the database from one valid state to another. It respects all rules, constraints, and integrity.



Example

A transfer keeps total account balance consistent (debit = credit).



In MySQL

Enforced through constraints (PRIMARY KEY, FOREIGN KEY, CHECK) and InnoDB engine.



I Isolation

Concurrent transactions do not interfere with each other. Each transaction appears to run independently.



Example

Two users updating data at the same time won't see each other's changes.



In MySQL

Handled using isolation levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE (default: REPEATABLE READ).



D Durability

Once a transaction is committed, its changes are permanent, even in case of system failure.



Example

After a successful transfer, data remains even if the server crashes.



In MySQL

InnoDB uses redo logs to permanently save committed changes.



ACID in Action (MySQL)

Use transactions to ensure ACID:

```
START TRANSACTION;  
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;  
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;  
COMMIT; -- or ROLLBACK; if any error occurs
```

Key Points

- ✓ InnoDB is the default storage engine that supports ACID.
- ✓ Always use transactions for reliable data operations.
- ✓ ACID ensures trust, reliability, and data integrity in MySQL.



CRUD OPERATIONS IN MYSQL

CRUD = Create, Read, Update, Delete



CREATE (INSERT)

Add new data into a table.

SYNTAX

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

EXAMPLE

```
INSERT INTO employees (id, name, department, salary)
VALUES (1, 'Aman', 'IT', 55000);
```

RESULT

A new row is inserted into the table.

id	name	department	salary
1	Aman	IT	55000



READ (SELECT)

Retrieve data from a table.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

EXAMPLE

```
-- Select all columns
SELECT * FROM employees;

-- Select specific columns
SELECT name, salary FROM employees;

-- Select with condition
SELECT * FROM employees
WHERE department = 'IT';
```

RESULT

Returns rows that match the query.

id	name	department	salary
1	Aman	IT	55000
2	Priya	HR	62000
3	Rohit	IT	58000



UPDATE

Modify existing data in a table.

SYNTAX

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

EXAMPLE

```
-- Increase salary of employee with id = 1
UPDATE employees
SET salary = 60000
WHERE id = 1;
```

RESULT

The existing row is updated.

id	name	department	salary
1	Aman	IT	60000
2	Priya	HR	62000
3	Rohit	IT	58000



DELETE

Remove data from a table.

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

EXAMPLE

```
-- Delete employee with id = 2
DELETE FROM employees
WHERE id = 2;
```

RESULT

The specified row is deleted.

id	name	department	salary
1	Aman	IT	60000
3	Rohit	IT	58000

★ KEY POINTS

- Always use WHERE clause in UPDATE and DELETE to avoid affecting all rows.
- Use SELECT before UPDATE or DELETE to verify data.
- Primary Key helps uniquely identify each row.

SAMPLE TABLE: employees

id (PK)	name	department	salary
1	Aman	IT	55000
2	Priya	HR	62000
3	Rohit	IT	58000

QUICK REFERENCE

INSERT	INTO table (...) VALUES (...);	-- Add new data
SELECT	... FROM table WHERE ...;	-- Read / Retrieve data
UPDATE	table SET ... WHERE ...;	-- Modify existing data
DELETE	FROM table WHERE ...;	-- Remove data



MySQL SELECT Statement – All Clauses

```
SELECT [DISTINCT] select_list
FROM table_name [AS alias]
[JOIN_TYPE] JOIN table_name2 [AS alias2] ON join_condition
WHERE where_condition
GROUP BY group_by_list
HAVING having_condition
ORDER BY order_by_list [ASC | DESC]
LIMIT row_count [OFFSET offset_row_count];
```

- 1. Columns to retrieve
- 2. Table to retrieve data from
- 3. Join another table
- 4. Filter rows
- 5. Group rows
- 6. Filter groups
- 7. Sort result
- 8. Limit result (and offset)

Explanation of All Clauses

Clause	Purpose	Description / Example	
1. SELECT	Select columns	Specifies the columns you want to retrieve.	SELECT id, name, salary
2. DISTINCT	Remove duplicates	Returns only distinct (unique) rows.	SELECT DISTINCT department
3. FROM	Specify table	Specifies the table to retrieve data from.	FROM employees e
4. JOIN ... ON	Join tables	Combines rows from two or more tables based on a related column.	INNER JOIN departments d ON e.dept_id = d.id
5. WHERE	Filter rows	Filters rows before grouping.	WHERE e.salary > 50000
6. GROUP BY	Group rows	Groups rows that have the same values.	GROUP BY d.name
7. HAVING	Filter groups	Filters groups after grouping.	HAVING COUNT(*) > 5
8. ORDER BY	Sort result	Sorts the result set.	ORDER BY e.salary DESC
9. LIMIT ... OFFSET	Limit result	Limits the number of rows returned and can skip rows.	LIMIT 10 OFFSET 20

JOIN Types

- INNER JOIN - Matches in both tables
- LEFT JOIN - All rows from left table
- RIGHT JOIN - All rows from right table
- LEFT JOIN - All rows from both tables

Complete Example

```
SELECT e.id, e.name, d.name AS department,
       COUNT(*) AS total_employees, AVG(e.salary) AS avg_salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id
WHERE e.status = 'Active'
GROUP BY d.name
HAVING COUNT(*) > 5
ORDER BY avg_salary DESC
LIMIT 10 OFFSET 0;
```



Execution Order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT/OFFSET

Built-in String Functions in MySQL

MySQL provides a rich set of built-in string functions to manipulate, format, and analyze text data.

Function	Description	Example	Result	Notes
 UPPER (str)	Converts a string to uppercase.	<code>SELECT UPPER('mysql string');</code>	'MYSQL STRING'	Converts all alphabetic characters to uppercase.
 LOWER (str)	Converts a string to lowercase.	<code>SELECT LOWER('MySQL STRING');</code>	'mysql string'	Converts all alphabetic characters to lowercase.
 CONCAT (str1, str2, ...)	Concatenates two or more strings.	<code>SELECT CONCAT('Hello', ' ', 'World');</code>	'Hello World'	Joins strings together.
 LENGTH (str)	Returns the length of the string in bytes.	<code>SELECT LENGTH('MySQL');</code>	5	Counts bytes. Use <code>CHAR_LENGTH()</code> for character count.
 CHAR_LENGTH (str)	Returns the number of characters in a string.	<code>SELECT CHAR_LENGTH('MySQL 😊');</code>	6	Counts characters, not bytes.
 SUBSTRING (str, pos, len)	Extracts a substring from a string.	<code>SELECT SUBSTRING('MySQL Tutorial', 1, 5);</code>	'MySQL'	Position starts at 1.
 INSTR (str, substr)	Returns the position of the first occurrence.	<code>SELECT INSTR('MySQL Tutorial', 'SQL');</code>	4	Returns 0 if not found.
 LTRIM (str)	Removes leading spaces from a string.	<code>SELECT LTRIM(' MySQL');</code>	'MySQL'	Removes only leading spaces.
 RTRIM (str)	Removes trailing spaces from a string.	<code>SELECT RTRIM('MySQL ');</code>	'MySQL'	Removes only trailing spaces.
 TRIM (str)	Removes leading and trailing spaces.	<code>SELECT TRIM(' MySQL ');</code>	'MySQL'	Removes both leading and trailing spaces.
 REPLACE (str, from, to)	Replaces all occurrences of a substring with another.	<code>SELECT REPLACE('banana', 'a', 'o');</code>	'bonono'	Case-sensitive replacement.
 SPLIT_STR (str, delim, pos)	Returns the substring from a string, based on delimiter.	<code>SELECT SUBSTRING_INDEX('a,b,c,d', ',', 2);</code>	'a,b'	Use <code>SUBSTRING_INDEX()</code> for splitting in MySQL.



Quick Tips

- String functions are case-sensitive.
- Use `CHAR_LENGTH()` to get the actual number of characters.
- MySQL strings can be enclosed in single quotes (' ') or double quotes (" ").
- Combine functions for powerful text manipulation.

Example: Combining Functions

```
SELECT UPPER( TRIM( SUBSTRING(' mysql tutorial ', 2, 12) ) ) AS result;
-- Extracts 'mysql tutori', trims spaces, and converts to uppercase
```



Result

'MYSQL TUTORI'



These built-in string functions help you clean, transform, and analyze text data efficiently in your MySQL queries.

Built-in Numerical Functions in MySQL

MySQL provides a rich set of built-in numerical functions to perform mathematical calculations, rounding, truncation, and other numeric operations.



1 Arithmetic Functions

Function	Description	Example	Result
ABS(x)	Returns the absolute value of x	<code>SELECT ABS(-25);</code>	25
CEIL(x) / CEILING(x)	Returns the smallest integer greater than or equal to x	<code>SELECT CEIL(4.3);</code>	5
FLOOR(x)	Returns the largest integer less than or equal to x	<code>SELECT FLOOR(4.7);</code>	4
ROUND(x, d)	Rounds x to d decimal places (d is optional)	<code>SELECT ROUND(23.567, 2);</code>	23.57
TRUNCATE(x, d)	Truncates x to d decimal places without rounding	<code>SELECT TRUNCATE(23.567, 2);</code>	23.56
MOD(x, y) / x % y	Returns the remainder of x divided by y	<code>SELECT MOD(10, 3);</code>	1

4 Rounding & Truncation Functions

Function	Description	Example	Result
ROUND(x)	Rounds x to the nearest integer	<code>SELECT ROUND(4.6);</code>	5
TRUNCATE(x)	Truncates x to the nearest integer	<code>SELECT TRUNCATE(4.9);</code>	4
SIGN(x)	Returns -1 if x < 0, 0 if x = 0, 1 if x > 0	<code>SELECT SIGN(-15);</code>	-1



Note:

- Most functions work with integer, decimal, and floating-point numbers.
- Use CAST() or CONVERT() if you need to convert strings to numbers before applying functions.

2 Power & Root Functions

Function	Description	Example	Result
POW(x, y) / POWER(x, y)	Returns x raised to the power y	<code>SELECT POW(2, 3);</code>	8
SQRT(x)	Returns the square root of x	<code>SELECT SQRT(16);</code>	4
CBRT(x)	Returns the cube root of x	<code>SELECT CBRT(27);</code>	3

3 Logarithmic & Exponential Functions

Function	Description	Example	Result
EXP(x)	Returns e raised to the power x (e ≈ 2.71828)	<code>SELECT EXP(1);</code>	2.71828...
LN(x)	Returns the natural logarithm of x	<code>SELECT LN(2.71828);</code>	1.00000...
LOG(x) / LOG10(x)	Returns the base-10 logarithm of x	<code>SELECT LOG10(1000);</code>	3

5 Other Useful Functions

Function	Description	Example	Result
RAND()	Returns a random floating-point number between 0 (inclusive) and 1 (exclusive)	<code>SELECT RAND();</code>	0.37482...
GREATEST(x1, x2, ...)	Returns the largest value	<code>SELECT GREATEST(4, 9, 2);</code>	9
LEAST(x1, x2, ...)	Returns the smallest value	<code>SELECT LEAST(4, 9, 2);</code>	2

</> Example Combined Query

```
SELECT
  ABS(-10) AS abs_val,
  ROUND(12.345, 2) AS rounded_val,
  POW(3, 2) AS power_val,
  SQRT(81) AS sqrt_val,
  MOD(20, 6) AS remainder;
```

abs_val	rounded_val	power_val	sqrt_val	remainder
10	12.35	9	9	2

Built-In Date & Time Functions in MySQL

MySQL provides a rich set of built-in functions to work with DATE, TIME, DATETIME, and TIMESTAMP values.

Category	Function	Description	Example	Result (Example)
 Current Date & Time	<code>CURDATE()</code>	Returns the current date.	<code>SELECT CURDATE();</code>	2025-05-29
	<code>CURTIME()</code>	Returns the current time.	<code>SELECT CURTIME();</code>	14:35:42
	<code>NOW()</code> / <code>SYSDATE()</code>	Returns the current date and time.	<code>SELECT NOW();</code>	2025-05-29 14:35:42
	<code>UTC_DATE()</code>	Returns the current UTC date.	<code>SELECT UTC_DATE();</code>	2025-05-29
	<code>UTC_TIME()</code>	Returns the current UTC time.	<code>SELECT UTC_TIME();</code>	08:05:42
	<code>UTC_TIMESTAMP()</code>	Returns the current UTC date and time.	<code>SELECT UTC_TIMESTAMP();</code>	2025-05-29 08:05:42
 Date Extraction	<code>YEAR(date)</code>	Extracts the year.	<code>SELECT YEAR('2025-05-29');</code>	2025
	<code>MONTH(date)</code>	Extracts the month (1-12).	<code>SELECT MONTH('2025-05-29');</code>	5
	<code>DAY(date)</code>	Extracts the day of month (1-31).	<code>SELECT DAY('2025-05-29');</code>	29
	<code>HOUR(time)</code>	Extracts the hour (0-23).	<code>SELECT HOUR('14:35:42');</code>	14
	<code>MINUTE(time)</code>	Extracts the minute (0-59).	<code>SELECT MINUTE('14:35:42');</code>	35
	<code>SECOND(time)</code>	Extracts the second (0-59).	<code>SELECT SECOND('14:35:42');</code>	42
 Date Calculation	<code>DATE_ADD(date, INTERVAL expr unit)</code>	Adds a time interval to a date.	<code>SELECT DATE_ADD('2025-05-29', INTERVAL 7 DAY);</code>	2025-06-05
	<code>DATE_SUB(date, INTERVAL expr unit)</code>	Subtracts a time interval from a date.	<code>SELECT DATE_SUB('2025-05-29', INTERVAL 1 MONTH);</code>	2025-04-29
	<code>DATEDIFF(date1, date2)</code>	Returns the number of days between two dates.	<code>SELECT DATEDIFF('2025-05-29', '2025-05-01');</code>	28
	<code>TIMESTAMPDIFF(unit, date1, date2)</code>	Returns the difference between two dates in the given unit.	<code>SELECT TIMESTAMPDIFF(DAY, '2025-05-01', '2025-05-29');</code>	28
 Date Formatting	<code>DATE_FORMAT(date, format)</code>	Formats a date as specified.	<code>SELECT DATE_FORMAT(NOW(), '%Y-%m-%d %H:%i:%s');</code>	2025-05-29 14:35:42
	<code>TIME_FORMAT(time, format)</code>	Formats a time as specified.	<code>SELECT TIME_FORMAT('14:35:42', '%h:%i %p');</code>	02:35 PM
 Other Useful Functions	<code>STR_TO_DATE(str, format)</code>	Converts a string to a date.	<code>SELECT STR_TO_DATE('29-05-2025', '%d-%m-%Y');</code>	2025-05-29
	<code>LAST_DAY(date)</code>	Returns the last day of the month.	<code>SELECT LAST_DAY('2025-05-29');</code>	2025-05-31
	<code>DAYNAME(date)</code>	Returns the day name (e.g., Monday).	<code>SELECT DAYNAME('2025-05-29');</code>	Thursday
	<code>WEEK(date, mode)</code>	Returns the week number.	<code>SELECT WEEK('2025-05-29', 1);</code>	22



Common Units for INTERVAL and TIMESTAMPDIFF

MICROSECOND, SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR

Quick Example

```
-- Add 10 days and format the result
SELECT DATE_FORMAT(
    DATE_ADD(CURDATE(), INTERVAL 10 DAY),
    '%Y-%m-%d'
) AS new_date; -- 2025-06-08
```

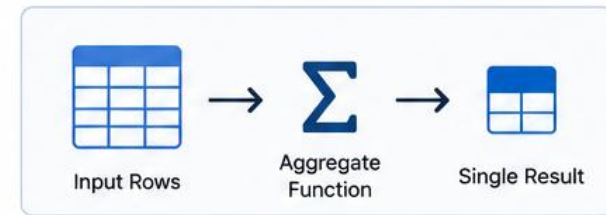


Tips

- MySQL date functions are not case-sensitive.
- Use single quotes for date/time literals (e.g., '2025-05-29').
- The default date format is 'YYYY-MM-DD'.
- All date functions work with DATE, DATETIME, and TIMESTAMP types.

Built-in Aggregate Functions in MySQL

Aggregate functions perform calculations on a set of values and return a single result. They are commonly used with **GROUP BY** to summarize data.



Function	Description	Example	Result (for values: 10, 20, 30, 40)	Notes
COUNT()	Returns the number of rows that match a specified condition.	<code>SELECT COUNT(*) FROM sales;</code>	4	COUNT(*) counts all rows. COUNT(column) counts non-NULL values in that column.
SUM()	Returns the sum of all non-NULL values.	<code>SELECT SUM(amount) FROM sales;</code>	100	Ignores NULL values.
AVG()	Returns the average of all non-NULL values.	<code>SELECT AVG(amount) FROM sales;</code>	25.00	Ignores NULL values.
MIN()	Returns the smallest non-NULL value.	<code>SELECT MIN(amount) FROM sales;</code>	10	Ignores NULL values.
MAX()	Returns the largest non-NULL value.	<code>SELECT MAX(amount) FROM sales;</code>	40	Ignores NULL values.
GROUP_CONCAT()	Returns a concatenated string of all non-NULL values.	<code>SELECT GROUP_CONCAT(name ORDER BY name SEPARATOR ', ') FROM employees;</code>	'John, Alice, Bob'	Useful for creating comma-separated lists. Order is optional.



Key Points

- Aggregate functions ignore **NULL** values (except **COUNT(*)**).
- They return a single value for a set of rows.
- Commonly used with **GROUP BY** to get summary per group.
- Can be used with **WHERE** to filter rows before aggregation.

Example with GROUP BY

```
SELECT department_id,  
       COUNT(*) AS total_employees,  
       AVG(salary) AS avg_salary,  
       SUM(salary) AS total_salary  
FROM   employees  
GROUP BY department_id  
ORDER BY department_id;
```



department_id	total_employees	avg_salary	total_salary
10	5	6200.00	31000.00
20	3	7200.00	21600.00
30	4	5500.00	22000.00



Aggregate functions help you analyze and summarize data efficiently, making them a powerful tool for reporting and decision-making.



Supported Data Types

COUNT(): Any data type
SUM(), AVG(), MIN(), MAX(): Numeric types
GROUP_CONCAT(): String types



Note

If no rows match the query:

- COUNT() returns 0
- Other aggregate functions return NULL

CASE STATEMENT IN MYSQL

The CASE statement in MySQL is used to perform conditional logic in queries. It works like IF-ELSE and returns values based on conditions.

1. SIMPLE CASE (Without Expression)

```
SELECT emp_id, emp_name, department,
CASE department
  WHEN 'HR' THEN 'Human Resources'
  WHEN 'IT' THEN 'Information Technology'
  WHEN 'FIN' THEN 'Finance'
  ELSE 'Other Department'
END AS dept_type
FROM employees;
```

2. SEARCHED CASE (With Conditions)

```
SELECT emp_id, emp_name, salary,
CASE
  WHEN salary >= 100000 THEN 'High Salary'
  WHEN salary >= 50000 THEN 'Medium Salary'
  WHEN salary >= 25000 THEN 'Low Salary'
  ELSE 'Very Low Salary'
END AS salary_category
FROM employees;
```

Clause	Description
CASE	Starts the CASE expression
WHEN	Checks a value
THEN	Result if condition is true
ELSE	Default result if no match
END	Ends the CASE expression
AS	Alias for the result column

How it works

- Evaluates conditions one by one.
- Returns the value of the first true condition.
- If none match, returns ELSE value.

3. CASE WITH AGGREGATE FUNCTIONS

```
SELECT department,
SUM(CASE
  WHEN gender = 'M' THEN 1
  ELSE 0
END) AS male_count,
SUM(CASE
  WHEN gender = 'F' THEN 1
  ELSE 0
END) AS female_count,
COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

4. NESTED CASE STATEMENT

```
SELECT emp_name, salary,
CASE
  WHEN department = 'IT' THEN
    CASE
      WHEN salary >= 100000 THEN 'IT - High'
      WHEN salary >= 60000 THEN 'IT - Medium'
      ELSE 'IT - Low'
    END
  WHEN department = 'HR' THEN
    CASE
      WHEN salary >= 80000 THEN 'HR - High'
      WHEN salary >= 50000 THEN 'HR - Medium'
      ELSE 'HR - Low'
    END
  ELSE 'Other Department'
END AS salary_level
FROM employees;
```

5. CASE WITH GROUP BY

```
SELECT
CASE
  WHEN salary >= 100000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_group,
COUNT(*) AS employee_count,
AVG(salary) AS avg_salary
FROM employees
GROUP BY
CASE
  WHEN salary >= 100000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END
ORDER BY
CASE salary_group
  WHEN 'High' THEN 1
  WHEN 'Medium' THEN 2
  ELSE 3
END;
```

Result Example

salary_group	employee_count	avg_salary
High	15	125000.00
Medium	28	65000.00
Low	12	32000.00

CASE can be used in:

- ✓ SELECT list
- ✓ WHERE clause
- ✓ ORDER BY clause
- ✓ GROUP BY clause
- ✓ HAVING clause
- ✓ UPDATE statement

6. REAL INTERVIEW EXAMPLES

1. Categorize students by marks

```
SELECT student_name, marks,
CASE
  WHEN marks >= 90 THEN 'A+'
  WHEN marks >= 75 THEN 'A'
  WHEN marks >= 60 THEN 'B'
  ELSE 'Low'
END AS grade;
```

2. Find active/inactive employees

```
SELECT emp_name, last_login,
CASE
  WHEN last_login >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
  THEN 'Active'
  ELSE 'Inactive'
END AS status;
```

3. Calculate bonus based on performance rating

```
SELECT emp_name, rating,
salary,
CASE rating
  WHEN 'Excellent' THEN salary * 0.20
  WHEN 'Good' THEN salary * 0.10
  WHEN 'Average' THEN salary * 0.05
  ELSE 0
END AS bonus
FROM employees;
```

7. PRACTICE QUESTIONS

1. Write a query to display employee name, salary and a column showing 'High' if salary > 70000, 'Medium' if salary between 40000 and 70000, otherwise 'Low'.
2. Write a query to count male and female employees in each department using CASE with aggregate functions.
3. Write a query to assign grades to students based on marks using searched CASE.
4. Write a query to show employees with a status column: 'Active' if they joined before 2 years, otherwise 'New'.
5. Write a query to group employees into experience buckets: '0-2 yrs', '2-5 yrs', '5+ yrs' using CASE in GROUP BY.

SYNTAX

Simple CASE
CASE expression
 WHEN value1 THEN result1
 WHEN value2 THEN result2
 ...
 ELSE else_result
END

Searched CASE
CASE
 WHEN condition1 THEN result1
 WHEN condition2 THEN result2
 ...
 ELSE else_result
END



- Tips:**
- CASE returns a single value.
 - Conditions are evaluated top to bottom.
 - Use ELSE to handle unmatched cases.
 - CASE is very useful for reporting and data analysis.



IF(), IFNULL(), COALESCE() in MySQL

Used for conditional logic and handling NULL values

1. IF() – Conditional Function

IF() returns one value if the condition is **TRUE** and another value if the condition is **FALSE**.

Syntax

```
IF(condition, value_if_true, value_if_false)
```

Example 1

Check salary and show status.

```
SELECT name, salary,
       IF(salary > 50000, 'High', 'Low') AS salary_status
FROM employees;
```

Result

name	salary	salary_status
Aman	55000	High
Priya	42000	Low
Rohit	75000	High
Neha	32000	Low

Example 2

Use IF() with multiple conditions (nested).

```
SELECT name, marks,
       IF(marks >= 90, 'A Grade',
          IF(marks >= 75, 'B Grade', 'C Grade')) AS grade
FROM students;
```

2. IFNULL() – Handle NULL Value

IFNULL() returns the first argument if it is **NOT NULL**. If it is **NULL**, it returns the second argument.

Syntax

```
IFNULL(expression, replacement_value)
```

Example 1

Replace **NULL** with a default value.

```
SELECT name, bonus,
       IFNULL(bonus, 0) AS bonus_amount
FROM employees;
```

Result

name	bonus	bonus_amount
Aman	5000	5000
Priya	NULL	0
Rohit	3000	3000
Neha	NULL	0

Example 2

Use IFNULL() in calculation.

```
SELECT name, salary, bonus,
       salary + IFNULL(bonus, 0) AS total_salary
FROM employees;
```

3. COALESCE() – Return First Non-NULL Value

COALESCE() returns the first **non-NULL** value from the list of expressions.

Syntax

```
COALESCE(value1, value2, value3, ...)
```

Example 1

Return the first available contact method.

```
SELECT name,
       COALESCE(phone, email, 'No Contact') AS contact
FROM customers;
```

Result

name	phone	email	contact
Aman	9876543210	aman@email.com	9876543210
Priya	NULL	priya@email.com	priya@email.com
Rohit	NULL	NULL	No Contact
Neha	9998887777	neha@email.com	9998887777

Example 2

Use COALESCE() with multiple columns.

```
SELECT name, COALESCE(mobile, home_phone, office_phone)
       AS contact_number
FROM clients;
```

KEY DIFFERENCE

IF() → Used for conditions (**TRUE** / **FALSE**).

Example: IF(salary > 50000, 'High', 'Low')

IFNULL() → Handles **NULL** for one expression.

Example: IFNULL(bonus, 0)

COALESCE() → Handles **NULL** for multiple expressions

Example: COALESCE(phone, email, 'No Contact')

Note:

Both **IFNULL()** and **COALESCE()** are used to handle **NULL** values.

Joins in MySQL

A **JOIN** clause is used to combine rows from two or more tables based on a related column between them.

Example Tables					
employees			departments		
id (PK)	name	department_id	id (PK)	department_name	
1	John	10	10	Sales	
2	Jane	20	20	HR	
3	Mike	10	30	IT	
4	Sara	30			

Key Points

- JOIN combines rows from two or more tables.
- Most joins require a matching condition using ON.
- INNER JOIN returns only matching rows.
- LEFT JOIN returns all rows from the left table.
- RIGHT JOIN returns all rows from the right table.
- FULL OUTER JOIN returns all rows from both tables.
- CROSS JOIN returns the Cartesian product.

Join Type	Description	Syntax	Example Query	Result
1. INNER JOIN	Returns only the rows that have matching values in both tables.	<pre>SELECT columns FROM table1 t1 INNER JOIN table2 t2 ON t1.common_column = t2.common_column;</pre>	<pre>SELECT e.id, e.name, d.department_name FROM employees e INNER JOIN departments d ON e.department_id = d.id;</pre>	<pre>+-----+-----+-----+ id name department_name +-----+-----+-----+ 1 John Sales 2 Jane HR 3 Mike Sales 4 Sara IT +-----+-----+-----+</pre>
2. LEFT JOIN	Returns all rows from the left table, and the matched rows from the right table. If no match, NULLs on the right side.	<pre>SELECT columns FROM table1 t1 LEFT JOIN table2 t2 ON t1.common_column = t2.common_column;</pre>	<pre>SELECT e.id, e.name, d.department_name FROM employees e LEFT JOIN departments d ON e.department_id = d.id;</pre>	<pre>+-----+-----+-----+ id name department_name +-----+-----+-----+ 1 John Sales 2 Jane HR 3 Mike Sales 4 Sara IT +-----+-----+-----+</pre>
3. RIGHT JOIN	Returns all rows from the right table, and the matched rows from the left table. If no match, NULLs on the left side.	<pre>SELECT columns FROM table1 t1 RIGHT JOIN table2 t2 ON t1.common_column = t2.common_column;</pre>	<pre>SELECT e.id, e.name, d.department_name FROM employees e RIGHT JOIN departments d ON e.department_id = d.id;</pre>	<pre>+-----+-----+-----+ id name department_name +-----+-----+-----+ 1 John Sales 3 Mike Sales 2 Jane HR 4 Sara IT +-----+-----+-----+</pre>
4. FULL OUTER JOIN*	Returns all rows when there is a match in either left or right table. If no match, NULLs on the unmatched side. (*MySQL doesn't support FULL OUTER JOIN directly.)	<pre>-- Simulate FULL OUTER JOIN in MySQL SELECT columns FROM table1 t1 LEFT JOIN table2 t2 ON t1.common_column = t2.common_column UNION SELECT columns FROM table1 t1 RIGHT JOIN table2 t2 ON t1.common_column = t2.common_column WHERE t1.common_column IS NULL;</pre>	<pre>-- Full outer join between employees and departments SELECT e.id, e.name, d.department_name FROM employees e LEFT JOIN departments d ON e.department_id = d.id UNION SELECT e.id, e.name, d.department_name FROM employees e RIGHT JOIN departments d ON e.department_id = d.id WHERE e.id IS NULL;</pre>	<pre>+-----+-----+-----+ id name department_name +-----+-----+-----+ 1 John Sales 2 Jane HR 3 Mike Sales 4 Sara IT +-----+-----+-----+</pre>
5. CROSS JOIN	Returns the Cartesian product of both tables (all combinations of rows).	<pre>SELECT columns FROM table1 t1 CROSS JOIN table2 t2;</pre>	<pre>SELECT e.name, d.department_name FROM employees e CROSS JOIN departments d;</pre>	<pre>+-----+-----+ name department_name +-----+-----+ John Sales John HR John IT Jane Sales Jane HR Jane HR +-----+-----+ (continues for all combinations)</pre>



Tips

- Always use table aliases for readability (e, d instead of employees, departments).
- Use INNER JOIN by default unless you specifically need unmatched rows.
- Add appropriate indexes on join columns for better performance.



Common Use Cases

- Combine related data (e.g., employees with departments).
- Report generation and analytics across multiple tables.
- Data validation and finding unmatched records (using LEFT/RIGHT JOIN).